

Predictive Analytics Using Historical Data

Submitted By:

Suraj Aggarwal

Course:

Data Analytics

Tools Used:

Microsoft Excel, Power Query, MySQL, Google Colab (Python), Power BI

Submission Date:

19 July 2026

Table of Contents

1. Introduction
2. Project Objective
3. Dataset Description
4. Tools Used
5. ETL Process
6. MySQL Database & SQL Analysis
7. Predictive Analytics
8. Model Evaluation
9. Power BI Dashboard
10. Conclusion

1. Introduction

The objective of this project is to develop a predictive analytics model using historical Walmart sales data. Historical sales, economic indicators, and store information were analyzed to understand sales patterns and predict future weekly sales using a Linear Regression model. The project demonstrates the complete analytics workflow, including data cleaning, database management, predictive modeling, and dashboard visualization.

2. Project Objective

- Analyze historical Walmart sales data.
- Clean and preprocess the dataset using Power Query.
- Store and analyze the data in MySQL.
- Build a Linear Regression model to predict weekly sales.
- Evaluate the prediction model using regression metrics.
- Create an interactive Power BI dashboard.

3. Dataset Description

Dataset Name

Walmart Historical Sales Dataset

Columns Used

- Store
- Department
- Date
- Weekly Sales
- Temperature
- Fuel Price
- CPI
- Unemployment
- Holiday Indicator

- Store Type
- Store Size
- Year
- Month
- Quarter
- Week of Year

Total Records

421,570 Rows

Total Columns

15 Columns

4. Tools Used

Tool	Purpose
Microsoft Excel	Data preprocessing
Power Query	ETL and feature engineering
MySQL	Database storage and SQL queries
Google Colab	Regression model and prediction
Python	Machine Learning
Power BI	Dashboard and visualization

5. ETL Process

Extract

Historical Walmart data was imported into Microsoft Excel.

Transform

The dataset was cleaned using Power Query.

The following transformations were performed:

- Removed null values
- Corrected data types
- Created Year column
- Created Month column
- Created Quarter column
- Created Week of Year column

Load

The cleaned dataset was exported as a CSV file and imported into MySQL.

6. MySQL Database & SQL Analysis

Query 1 – Total Number of Records

Objective: To verify that the dataset has been imported successfully into the MySQL database and to determine the total number of records available for analysis.

```
SELECT COUNT(*) AS Total_Records  
FROM walmart_sales;
```

- **421,570**

Query 2- Total Weekly Sales

Objective: To calculate the overall weekly sales generated across all Walmart stores during the selected time period.

```
SELECT SUM(Weekly_Sales) AS Total_Sales  
FROM walmart_sales;
```

- **6,737,215,014.45 (about \$6.74 billion).**

Query 3- Average Weekly Sales

Objective: To determine the average weekly sales value, providing an overview of typical sales performance.

```
SELECT AVG(Weekly_Sales) AS Average_Sales  
FROM walmart_sales;
```

- **15981.26**

Query 4- Top 10 Stores by Sales

Objective: To identify the top 10 stores with the highest total sales, helping evaluate the best-performing stores.

```
SELECT Store, SUM(Weekly_Sales) AS Total_Sales  
FROM walmart_sales
```

GROUP BY Store

ORDER BY Total_Sales DESC

LIMIT 10;

Rank Store Total Sales

1	20	301,397,792.46
2	4	299,543,953.38
3	14	288,999,911.34
4	13	286,517,703.80
5	2	275,382,440.98
6	10	271,617,713.89
7	27	253,855,916.88
8	6	223,756,130.64
9	1	222,402,808.85
10	39	207,445,542.47

Query 5- Monthly Sales Trend

Objective: To analyze monthly sales performance and identify seasonal trends or fluctuations in sales throughout the year.

SELECT Month, SUM(Weekly_Sales) AS Total_Sales

FROM walmart_sales

GROUP BY Month

ORDER BY Month;

Month	Total Sales (Approx.)
1	426,426,000
2	568,727,000
3	588,963,000
4	646,860,000
5	592,785,000
6	622,743,000
7	650,001,000
8	642,310,000
9	615,370,000
10	602,204,000
11	665,931,000
12	515,895,000

Query 6- Holiday vs Non-Holiday Sales

Objective: To compare total sales during holiday and non-holiday periods and assess the impact of holidays on sales performance.

```
SELECT Is_Holiday, SUM(Weekly_Sales) AS Total_Sales
FROM walmart_sales
GROUP BY Is_Holiday;
```


Is_Holiday	Total_Sales (Approx.)
0 (Non-Holiday)	6,231,919,434.89
1 (Holiday)	505,295,579.56

Query 7- Department-wise Sales

Objective: To identify the departments that contribute the most to overall sales and evaluate departmental performance.

```
SELECT Department, SUM(Weekly_Sales) AS Total_Sales
FROM walmart_sales
GROUP BY Department
ORDER BY Total_Sales DESC
LIMIT 10;
```

Rank	Department	Total Sales (Approx.)
1	92	483,943,295.06
2	95	449,758,398.07
3	90	387,786,373.57
4	72	305,245,672.11
5	38	303,908,891.97
6	2	249,450,580.41
7	40	228,542,601.45
8	91	220,959,313.29

Rank	Department	Total Sales (Approx.)
9	8	205,007,415.73
10	13	194,760,376.28

Query 8- Maximum Weekly Sale

Objective: To determine the highest weekly sales recorded in the dataset and identify the peak sales performance.

```
SELECT MAX(Weekly_Sales) AS Highest_Weekly_Sale
FROM walmart_sales;
```

- 693099.36

7. Predictive Analytics

The objective of this phase was to develop a machine learning model capable of predicting **Weekly Sales** using historical Walmart sales data and economic indicators. A **Linear Regression** algorithm was selected because it is suitable for predicting continuous numerical values and provides an interpretable relationship between input variables and the target variable.

Before model development, the cleaned dataset obtained from the ETL process was imported into **Google Colab** using the Pandas library. Missing values were checked and handled, and only the relevant features were selected for training the model.

The following independent variables were used as input features:

- Temperature
- Fuel Price

- Consumer Price Index (CPI)
- Unemployment Rate
- Store Size

The dependent (target) variable was:

- **Weekly Sales**

The dataset was divided into **80% training data** and **20% testing data** using the `train_test_split()` function from the Scikit-learn library. The training dataset was used to train the Linear Regression model, while the testing dataset was used to evaluate the model's predictive performance.

After training, the model generated predictions for the testing dataset. The predicted values were compared with the actual weekly sales to measure the accuracy and effectiveness of the model.

```
[36] df.info()
✓ 0s

... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 421570 entries, 0 to 421569
Data columns (total 15 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   Store            421570 non-null  int64
1   Department       421570 non-null  int64
2   Date             421570 non-null  object
3   Weekly_Sales     421570 non-null  float64
4   Is_Holiday       421570 non-null  bool
5   Temperature      421570 non-null  float64
6   Fuel_Price       421570 non-null  float64
7   CPI              421570 non-null  float64
8   Unemployment     421570 non-null  float64
9   Type             421570 non-null  object
10  Size             421570 non-null  int64
11  Year             421570 non-null  int64
12  Month            421570 non-null  int64
13  Quarter          421570 non-null  int64
14  Week_of_Year     421570 non-null  int64
dtypes: bool(1), float64(5), int64(7), object(2)
memory usage: 45.4+ MB
```

```
[211] coefficients = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_
})

coefficients
```

	Feature	Coefficient
0	Temperature	27.956012
1	Fuel_Price	-499.187831
2	CPI	-18.728528
3	Unemployment	-260.372714
4	Size	0.090162

Next steps: [Generate code with coefficients](#) [New interactive sheet](#)

```
[145] comparison = pd.DataFrame({
    "Actual": y_test,
    "Predicted": predictions
})

comparison.head(10)
```

	Actual	Predicted
272342	50932.42	21530.067385
176581	3196.12	14257.839840
354212	10125.03	6615.184560
281444	3311.26	12095.050667
124208	6335.65	24174.208180
123901	8971.23	24902.322139
111563	1575.35	13650.695612
93836	17308.45	16120.994924
102266	5763.39	22186.010076
254986	17034.57	18210.369792

Next steps: [Generate code with comparison](#) [New interactive sheet](#)

```
[316]
✓ Os ▶ import pandas as pd

coefficients = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.coef_
})

coefficients = coefficients.sort_values(by='Coefficient', ascending=False)

print(coefficients)
```

```
***
   Feature  Coefficient
0  Temperature  27.956012
4      Size      0.090162
2      CPI    -18.728528
3  Unemployment -260.372714
1  Fuel_Price -499.187831
```

```
[365]
✓ Os ▶ print("===== MODEL PERFORMANCE =====")
print("Mean Absolute Error :", mae)
print("Mean Squared Error :", mse)
print("Root Mean Squared Error :", rmse)
print("R2 Score :", r2)
```

```
*** ===== MODEL PERFORMANCE =====
Mean Absolute Error : 14517.243870422344
Mean Squared Error : 488937653.9443294
Root Mean Squared Error : 22111.934649512903
R2 Score : 0.06245747548727698
```

Machine Learning Model

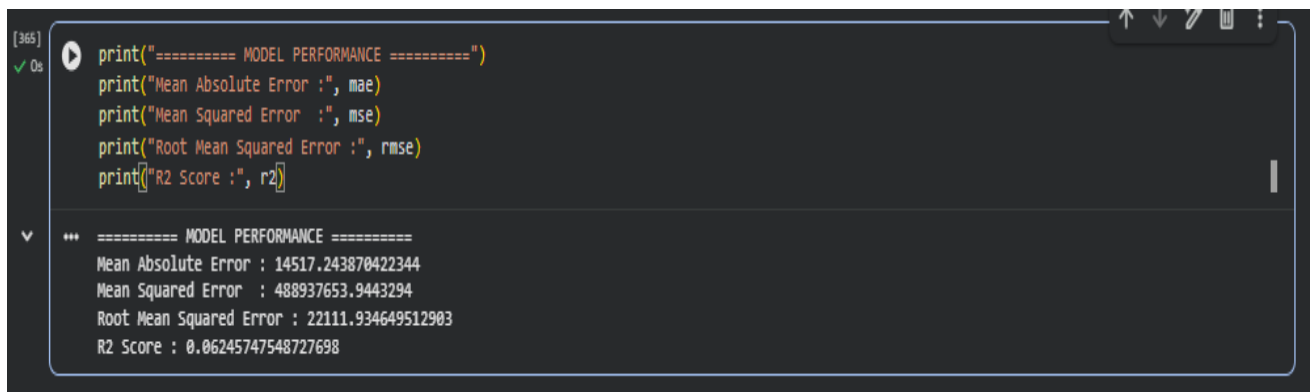
Item	Details
Algorithm	Linear Regression
Programming Language	Python
Development Environment	Google Colab
Library Used	Scikit-learn
Training Data	80%
Testing Data	20%
Target Variable	Weekly_Sales
Input Features	Temperature, Fuel_Price, CPI, Unemployment, Size

8. Model Evaluation

After training the Linear Regression model, its performance was evaluated using the testing dataset. The predicted weekly sales were compared with the actual weekly sales to measure the accuracy of the model. Several standard regression evaluation metrics were used to assess the prediction performance.

The following metrics were calculated using the Scikit-learn library:

Evaluation Metric	Value
Mean Absolute Error (MAE)	14517.243870422344
Mean Squared Error (MSE)	488937653.9443294
Root Mean Squared Error (RMSE)	22111.934649512903
R ² Score	0.06245747548727698

A screenshot of a Jupyter Notebook cell. The top part shows the code being executed:

```
print("===== MODEL PERFORMANCE =====")
print("Mean Absolute Error :", mae)
print("Mean Squared Error :", mse)
print("Root Mean Squared Error :", rmse)
print("R2 Score :", r2)
```

 The bottom part shows the output of the code:

```
*** ===== MODEL PERFORMANCE =====
Mean Absolute Error : 14517.243870422344
Mean Squared Error : 488937653.9443294
Root Mean Squared Error : 22111.934649512903
R2 Score : 0.06245747548727698
```

Interpretation of Evaluation Metrics

Mean Absolute Error (MAE)

MAE represents the average difference between the actual weekly sales and the predicted weekly sales. A lower MAE indicates that the model's predictions are closer to the actual values.

Mean Squared Error (MSE)

MSE measures the average of the squared prediction errors. It gives greater importance to larger prediction errors. A lower MSE indicates better model performance.

Root Mean Squared Error (RMSE)

RMSE is the square root of the Mean Squared Error and represents the average prediction error in the same unit as Weekly Sales. Lower RMSE values indicate more accurate predictions.

R² Score

The R² Score measures how well the model explains the variation in weekly sales. An R² value closer to **1** indicates a better fit, while a value closer to **0** indicates weaker predictive performance.

Prediction Results

The model generated predictions for the testing dataset using the trained Linear Regression algorithm. These predictions were compared with the actual weekly sales values to evaluate the effectiveness of the predictive model.

Visualizations Included:

1. **Actual vs Predicted Scatter Plot**
2. **Actual vs Predicted Line Chart**
3. **Residual Plot**
4. **Feature Importance (Regression Coefficients)**



Figure 1

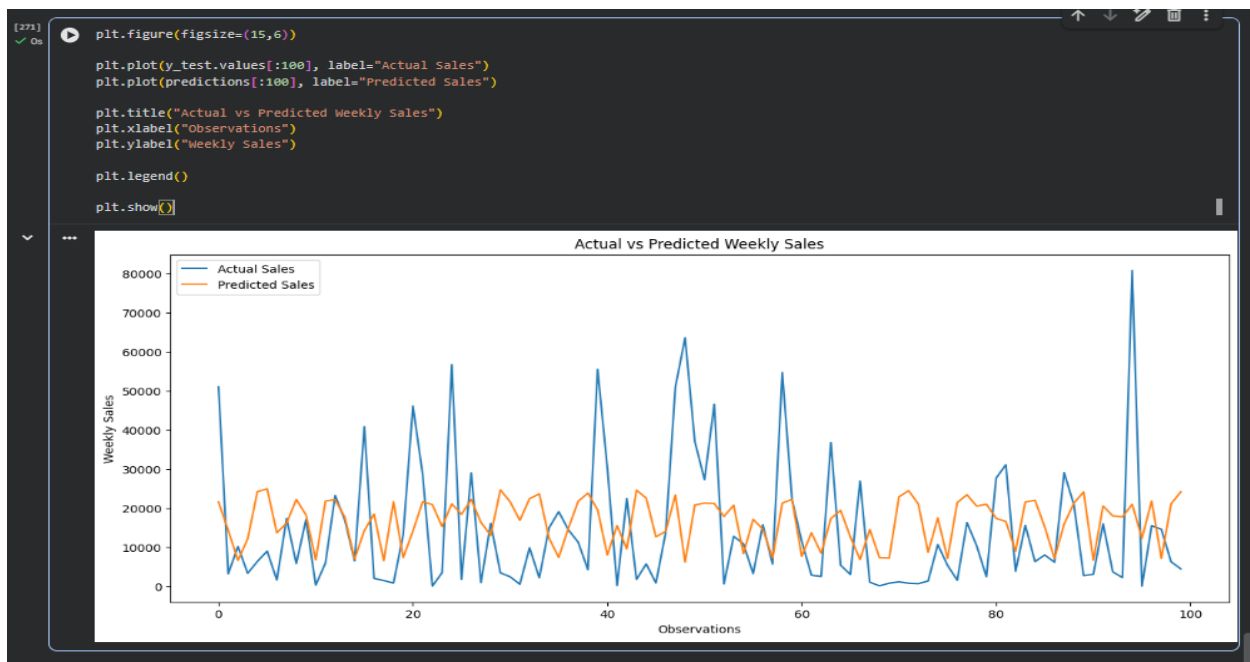


Figure 2



Figure 3

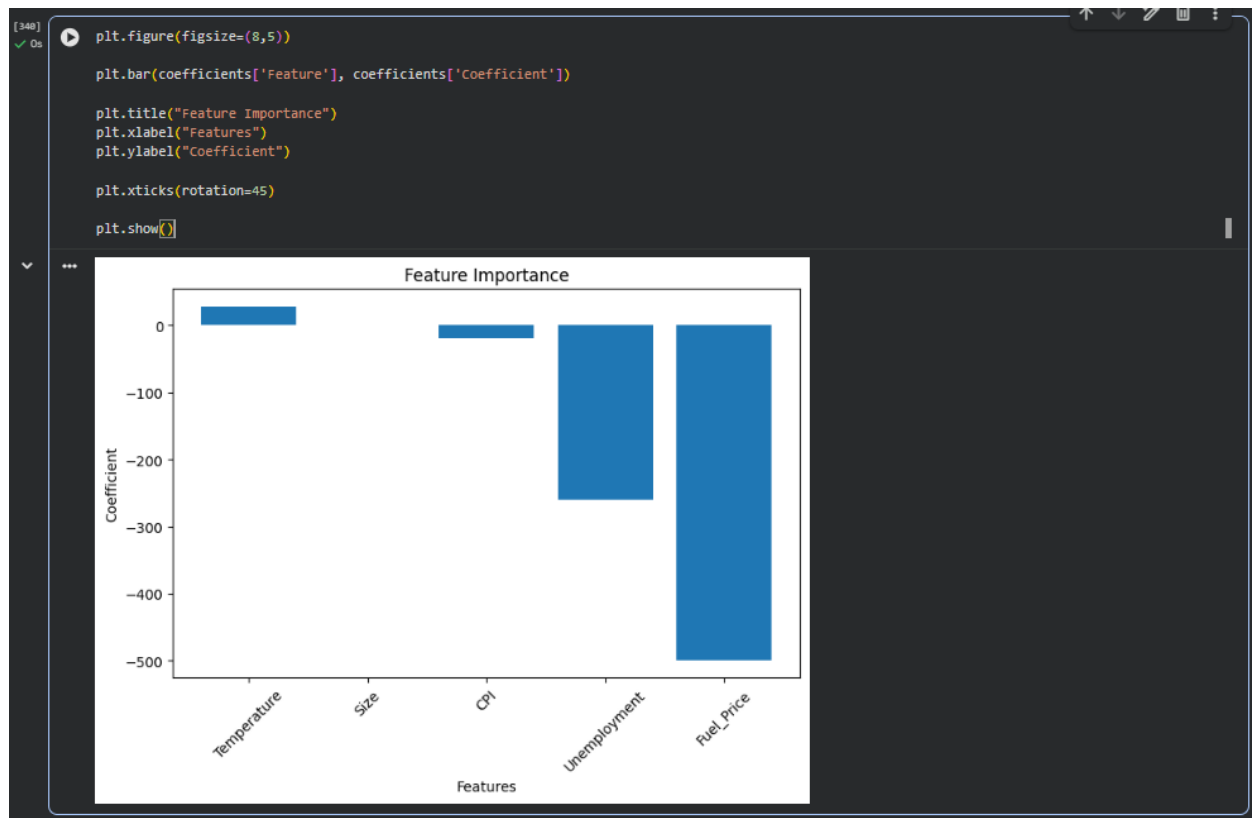


Figure 4

Explanation:

Figure 1: Actual vs Predicted Weekly Sales Scatter Plot showing the relationship between observed and predicted sales values.

Figure 2: Actual vs Predicted Line Chart comparing actual sales with model predictions.

Figure 3: Residual Plot illustrating the prediction errors around the zero baseline.

Figure 4: Feature Importance chart showing the influence of each independent variable on Weekly Sales.

Overall Model Performance

The Linear Regression model successfully predicted weekly sales using historical sales data and economic indicators. The evaluation metrics and visualizations indicate that the model captures the relationship between the selected features and weekly sales. The predictive model provides useful insights into future sales trends and supports data-driven decision-making.

9. Power BI Dashboard

An interactive Power BI dashboard was developed to visualize historical Walmart sales data and present key business insights in a user-friendly format. The dashboard enables users to monitor sales performance, compare stores and departments, analyze sales trends, and filter the data dynamically using interactive slicers.

The dashboard was designed using the cleaned dataset generated during the ETL process and includes key performance indicators (KPIs), charts, and filters to support data-driven decision-making.

Dashboard Components

KPI Cards

The dashboard includes the following KPI cards:

- Total Sales
- Average Sales
- Maximum Sales
- Minimum Sales
- Total Stores
- Total Departments

These KPIs provide a quick summary of the overall business performance.

Charts and Visualizations

The following visualizations were created:

Visualization

Purpose

Monthly Sales Trend (Line Chart)	Analyzes sales trends across different months.
Sales by Store (Column Chart)	Compares sales performance among Walmart stores.
Sales by Department (Bar Chart)	Identifies the highest-performing departments.
Sales Distribution by Store Type (Pie Chart)	Shows the contribution of each store type to total sales.
Quarter-wise Sales (Column Chart)	Compares sales performance across quarters.

Interactive Filters (Slicers)

The dashboard includes interactive slicers for:

- Year
- Month
- Store
- Store Type

These filters allow users to explore specific segments of the data and perform detailed analysis.

Key Insights

- The dashboard provides a comprehensive overview of Walmart's historical sales performance.
- Monthly and quarterly charts help identify sales trends and seasonal variations.
- Store and department comparisons highlight the highest-performing business units.
- Scatter plots illustrate the relationship between economic factors (temperature, fuel price, and unemployment) and weekly sales.
- Interactive filters make it easy to analyze sales data for specific time periods, stores, or store types.

Walmart Sales Predictive Analytics Dashboard

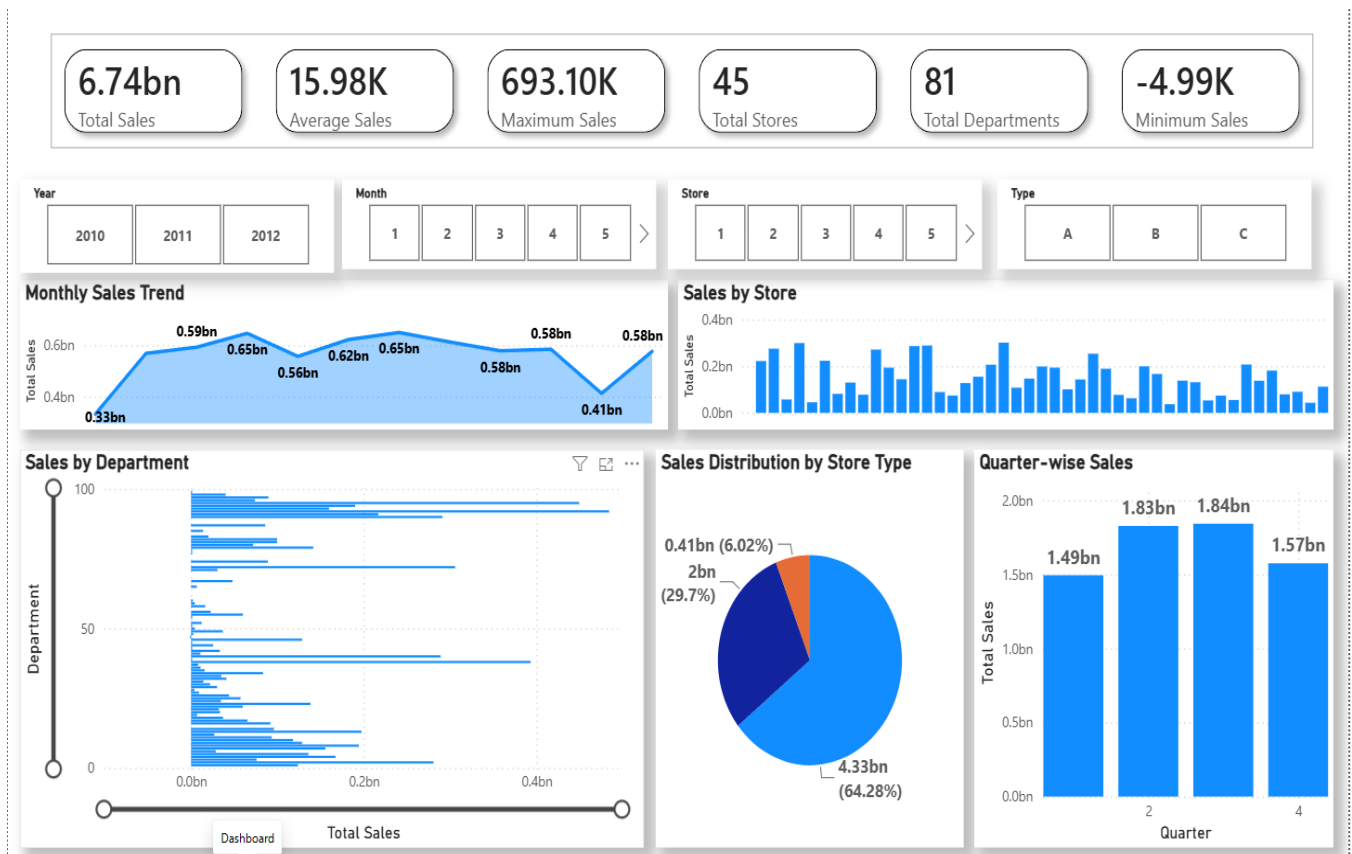
Subject: Data Analytics

Project: Predictive Analytics Using Historical Data

Dataset: Walmart Historical Sales Dataset

Student Name: Suraj Aggarwal

Submission Date: 19 July 2026



Dashboard Summary

The Power BI dashboard transforms raw sales data into meaningful visual insights by combining KPI cards, interactive charts, and slicers. It enables users to monitor business performance, identify sales patterns, and support informed decision-making through clear and interactive data visualization.

10. Conclusion

This project successfully implemented predictive analytics using historical Walmart sales data. The data was cleaned and transformed through Power Query, analyzed using SQL, and used to train a Linear Regression model for predicting weekly sales. Model performance was evaluated using standard regression metrics, and an interactive Power BI dashboard was developed to present key business insights. The project demonstrates practical skills in data preprocessing, database analysis, machine learning, and business intelligence.